Survey of Randomized Neural Network Methods for Solving Partial Differential Equations: A Comparative Review of Three Recent Advances

Your Name

May 2, 2025

Abstract

This survey reviews and compares three recent advances in randomized neural network (RNN) methods for solving partial differential equations (PDEs): the Extreme Learning Machine (ELM) approach for high-dimensional PDEs [P1], the Randomized Neural Network with Petrov–Galerkin (RNN-PG) method [P2], and the application of local RNNs to linear elasticity and Navier–Stokes equations [P3]. We summarize their theoretical foundations, algorithmic frameworks, and numerical results, and discuss their practical implications for scientific computing.

1 Introduction

The use of neural networks for solving PDEs has seen rapid development, especially with the advent of randomized neural network (RNN) approaches. These methods aim to overcome the curse of dimensionality and improve computational efficiency by leveraging randomization in network parameters. This survey focuses on three representative works: ELM for high-dimensional PDEs [P1], RNN-PG for general linear and nonlinear PDEs [P2], and local RNNs for coupled systems [P3].

2 Extreme Learning Machine for High-Dimensional PDEs [P1]

2.1 Method Overview

The ELM approach represents the solution to a PDE as a single-hidden-layer feedforward neural network (FNN), where the hidden-layer weights and biases are randomly assigned and fixed, and only the output weights are trained. The solution $u(x)$ is approximated as:

$$u(x) = \sum_{j=1}^{N_u} \phi_j V_j(x), \tag{1}$$

where $V_j(x)$ are the outputs of the hidden layer with random parameters, and ϕ_j are the trainable output weights. The PDE and boundary/initial conditions are enforced at random collocation points in the domain and on the boundary. For linear PDEs, this leads to a linear least squares problem for ϕ_j; for nonlinear PDEs, a nonlinear least squares solver (e.g., Gauss-Newton) is used. The method can be extended to a local ELM (locELM) variant, where the domain is decomposed along up to $\mathcal{M}$ directions, and each subdomain is assigned a local ELM network. Continuity conditions are enforced at shared subdomain boundaries.

A second method, ELM/A-TFC, combines ELM with an approximate Theory of Functional Connections (A-TFC) to systematically enforce boundary/initial conditions via a constrained expression. The free function in the A-TFC form is represented by an ELM network and trained analogously.

2.2 Theoretical Results

ELM-type randomized neural networks are universal function approximators. For any continuous function f on $[0,1]^d$ satisfying a Lipschitz condition, there exists a randomized NN approximation $f_{\omega_n}(x)$ such that:

$$\mathbb{E} \int_K |f(x) - f_{\omega_n}(x)|^2 dx \leq \frac{C}{n}, \tag{2}$$

where n is the number of basis functions, C is a constant independent of n, and the expectation is over the random parameters. Notably, the convergence

rate is independent of the input dimension d in expectation, in contrast to deterministic basis expansions. The paper also discusses the impact of the randomization range R_m for hidden-layer parameters, which must be tuned for each problem and generally decreases with increasing dimension.

For the ELM/A-TFC method, the A-TFC constrained expression reduces the exponential growth in the number of terms for high-dimensional problems, at the cost of only approximately enforcing boundary conditions. The conditions for the free function in A-TFC are simpler than the original boundary conditions.

2.3 Numerical Experiments

The ELM and ELM/A-TFC methods are tested on a variety of high-dimensional linear and nonlinear, stationary and time-dependent PDEs, including Poisson, nonlinear Poisson, advection-diffusion, and KdV equations, up to 15 dimensions. The results show quasi-exponential error decay with respect to the number of network parameters and boundary collocation points, with errors reaching near machine precision in lower dimensions. The number of interior collocation points has little effect on accuracy in high dimensions. Compared to PINNs, ELM achieves much higher accuracy and lower computational cost. The local ELM variant is effective for problems with local features.

3 Randomized Neural Network with Petrov–Galerkin Methods [P2]

3.1 Method Overview

The RNN-PG method approximates the solution to a PDE using a randomized neural network as the trial space and a Petrov–Galerkin variational formulation. The neural network $u_\rho(x)$ is constructed with all weights and biases randomly initialized and fixed except for the last layer, whose weights are trainable. The trial space U_ρ is the span of the outputs of the last hidden layer, and the degrees of freedom are the number of neurons in that layer. The test space V_h can be chosen flexibly (e.g., finite element, polynomial, or

neural network bases). The variational form is:

$$a(u_\rho, v_h) = l(v_h), \quad \forall v_h \in V_h, \tag{3}$$

with Dirichlet boundary conditions enforced at random points on the boundary. This leads to a (possibly overdetermined) linear system, solved in the least squares sense. The method is mesh-free and can handle various boundary conditions and problem types.

The method is extended to mixed formulations (M-RNN-PG) for systems with multiple unknowns, such as the Poisson equation in mixed form, by using separate neural networks for each variable and constructing the appropriate variational forms.

For time-dependent problems, a space–time approach is used, treating time as an additional input variable and enforcing initial and boundary conditions at random points in the space–time domain.

3.2 Theoretical Results

The RNN-PG method leverages the universal approximation property of neural networks. Theoretical error bounds are available for neural networks with certain activation functions (e.g., tanh, logistic), showing that for any f in a Sobolev space $W^{s,p}$, there exists a neural network f_ρ with depth $D \leq C \log(d+s)$ and number of nonzero weights $M_D \leq C\epsilon^{-d/s-k-\mu_k}$ such that:

$$\|f - f_\rho\|_{W^{k,p}} \leq \epsilon. \tag{4}$$

The accuracy of the RNN-PG method depends on the expressiveness of the trial space U_ρ, the choice of test functions, and the number of collocation points for boundary conditions. Unlike standard Petrov–Galerkin methods, the RNN-PG method does not require verification of the inf-sup condition, as the least squares solution always exists.

For mixed formulations, the method avoids the technical requirements of choosing compatible finite element pairs, as the least squares approach ensures solvability.

3.3 Numerical Experiments

The RNN-PG method is demonstrated on elliptic, parabolic, and nonlinear PDEs, including high-dimensional and time-dependent problems. Examples

include 2D Poisson, heat, wave, advection-diffusion, and Burgers' equations, as well as high-dimensional heat equations. The method achieves high accuracy (e.g., L^2 errors $< 10^{-8}$) with relatively few degrees of freedom, and outperforms finite element methods in terms of accuracy per degree of freedom. The choice of test functions (e.g., finite element, neural network, polynomial) affects the conditioning and accuracy of the linear system. The method is robust to the number of interior collocation points and benefits from deeper networks and more test functions. For nonlinear problems, Picard and Newton iterations are used.

4 Local RNNs for Linear Elasticity and Navier–Stokes [P3]

4.1 Method Overview

This work extends the RNN-PG framework to coupled systems such as linear elasticity and Navier–Stokes equations, using local domain decomposition. The domain is partitioned into subdomains, each approximated by a local RNN. Interface conditions are enforced at subdomain boundaries to ensure global consistency. Mixed formulations are used for coupled variables, and the least squares approach is used to solve the resulting system. The method supports both primal and mixed formulations, and can handle solid-fluid interaction, incompressible flow, and large deformation problems.

4.2 Theoretical Results

The paper analyzes the stability and error of the coupled system, showing that local RNNs can accurately capture complex interactions and interface conditions. The least squares approach ensures well-posedness without strict compatibility conditions on subdomain bases. The method is flexible in the choice of test functions and robust to the number of subdomains and network architectures. Theoretical results indicate that the method can achieve high accuracy and stability for a wide range of coupled PDE systems.

4.3 Numerical Experiments

Numerical results include solid-fluid interaction, incompressible flow, and large deformation problems. The local RNN approach achieves high accuracy, efficient parallelization, and robust handling of complex geometries, outperforming traditional FEM in some cases. The method is demonstrated on benchmark problems in linear elasticity and Navier–Stokes, showing rapid error decay and efficient use of computational resources.

5 Comparative Discussion

All three methods exploit randomization in neural network architectures to reduce training cost and improve scalability. ELM [P1] is particularly effective for high-dimensional problems, while RNN-PG [P2] offers flexibility in test function choice and variational formulations. The local RNN approach [P3] extends these ideas to multiphysics and coupled systems, enabling domain decomposition and parallelism. Compared to traditional methods, these RNN-based solvers are mesh-free, easily handle complex boundaries, and can achieve high accuracy with fewer parameters. However, challenges remain in network initialization, conditioning, and adaptivity for very complex or highly nonlinear problems.

6 Conclusion

Randomized neural network methods represent a promising direction for scientific machine learning and PDE solvers. The ELM, RNN-PG, and local RNN approaches reviewed here demonstrate strong theoretical foundations, practical efficiency, and broad applicability. Future work may focus on adaptive architectures, improved theoretical analysis, and large-scale parallel implementations.

textbfReferences

[P1] Y. Wang, S. Dong, "An extreme learning machine-based method for computational PDEs in higher dimensions," Comput. Methods Appl. Mech. Engrg., 2024.

[P2] Y. Shang, F. Wang, J. Sun, "Randomized neural network with Petrov–Galerkin methods for solving linear and nonlinear partial differential equations," Commun. Nonlinear Sci. Numer. Simul., 2023.

[P3] Y. Shang, F. Wang, J. Sun, "Randomized neural network with Petrov–Galerkin methods for solving linear elasticity and Navier–Stokes equations," Commun. Nonlinear Sci. Numer. Simul., 2023.